

```
ALTER TABLE product_items_j ADD CONSTRAINT chk_item_name CHECK
(item_name LIKE 'PK');
```

More importantly, the *WHERE* condition should be similar to the *CHECK* constraints. For example, if you have the following *CHECK* constraint:

```
ALTER TABLE product_items_j ADD CONSTRAINT chk_item_name CHECK
(item_name BETWEEN 'P' AND 'PB');
```

Then constraint exclusion would not help for the following query, since the *WHERE* condition is not similar to the *CHECK* constraint:

```
SELECT item_name FROM product_items WHERE item_name LIKE 'Q%';
```

To get the constraint exclusion working, you need this form of query:

```
SELECT item_name FROM product_items WHERE item_name = 'Pen';
```

From the above examples, it should be clear that the user needs to take extra effort while handling the *WHERE* clause predicates, in order to hit the *CHECK* constraints. The other major problem with this set-up is that there is no automatic way to verify if all the *CHECK* constraints are mutually exclusive. You need to be extra careful while setting them up.

The *EXPLAIN* feature comes in very handy to tackle these issues. Basically, any change to the query or table, even the slightest one, should be rigorously followed by looking at the *EXPLAIN* output. If you do not see the expected plan, then either the *WHERE* clause or the *CHECK* constraints need to be looked into.

Inheritance

The important rule that you should always be aware of is that child tables inherit column *DEFAULT* values, *NOT NULL* and *CHECK* constraints only. Any other constraints, like *UNIQUE*, *PRIMARY* and *FOREIGN* key, will not be inherited. Also, indexes, ownership and permissions will not be inherited.

One has to set up the non-inherited constraints on all the child tables. However, there are various ways of setting up these constraints, which we will discuss in the very next section. The child tables cannot rename inherited columns, but can add new columns—this is the biggest advantage of using inheritance.

You should also note that one can enable or disable inheritance on child tables by using the *ALTER TABLE* command. This is, in fact, very useful when you want to remove or move partitions from the base table.

Using inheritance, you can have multiple levels of

partitioning. You can always create a great-great-grandchild of a partition, but you might experience a performance loss with this kind of set-up, since more base tables are involved, directly or indirectly. So generally, 1 or 2 levels for partitions and sub-partitions should be good. If you are going any deeper, then most probably you need to re-think the partitioning scheme employed.

Uniqueness

PostgreSQL uses indexes for supporting uniqueness. However, there are no multi-table indexes, which means that an index cannot span over more than one table. This is the reason for not having any primary key in our set-up. In general, you might want to concentrate on indexing the partitioning key columns. For this, create non-overlapping *CHECK* constraints and then create a unique index on partitioning key columns over each partition. This is almost like having unique values over all partitions.

For indexing the non-partitioning-key columns, you create unique indexes over each partition. However, for verifying uniqueness, you need custom functions to scan all partitions, which would hopefully use the related indexes. As you must have realised, this scanning of all partitions was not what we wanted, to begin with. Also, due to the lack of multi-table indexes, maintaining a primary key or a foreign key is technically not possible. Consider that you created a primary key on each of the *order_id* columns of all child tables. Now it may happen that both *orders_part_2011* and *order_part_2010* have an *order_id* value of 1. Even though the uniqueness constraint within each child table is upheld, the base table orders now has duplicate *order_id* values.

To overcome this, you can create an additional lookup table, with primary keys from the base as well as child tables. Basically, you need to use triggers to keep the lookup table updated as you insert, update or delete records from the related partitioned table. Now all the primary and foreign key constraints can be mapped to this lookup table.

That's it! These are basic set-up issues that you must be able to get through now. You should now be ready to face the next challenge—performance tuning for partitions.

What's around the corner?

The PostgreSQL community does have several discussions on this feature. For more details, you can visit http://wiki.postgresql.org/wiki/Table_partitioning. 

By: Chetan Sutraway

Chetan is a software engineer working at EnterpriseDB. He has 9 years of experience in DBMS internals development, with expertise in query optimisation. His other interests include cloud computing and contributing to open source projects. He can be reached at chetan.sutraway@enterprisedb.com.